

Empirical Evaluation of Parameter-Efficient Fine-Tuning for Text Classification

A Comparative Study of Full Fine-Tuning, LoRA, and QLoRA

Dev Kumar

Abstract

This technical case study presents an empirical comparison of three strategies for adapting a pretrained transformer model to a downstream text-classification task: Full Fine-Tuning, Low-Rank Adaptation (LoRA), and Quantized Low-Rank Adaptation (QLoRA). The experiments use the `distilbert-base-uncased` model on the Stanford IMDB movie-review sentiment classification dataset. To evaluate the computational and predictive trade-offs among the three approaches, the study measures trainable parameter count, peak GPU memory consumption, wall-clock training time, test accuracy, and test F1 score.

The experimental results show substantial differences in resource requirements. Full Fine-Tuning updates 66,955,010 parameters, whereas LoRA and QLoRA update only 739,586 parameters. Peak GPU memory consumption is reduced from 1,720.34 MB for Full Fine-Tuning to 995.70 MB for LoRA and 966.32 MB for QLoRA. LoRA records the shortest measured training time at 61.40 seconds, compared with 105.76 seconds for Full Fine-Tuning and 85.26 seconds for QLoRA. The corresponding test accuracies are 88.60%, 87.20%, and 86.70%, respectively.

The results demonstrate that parameter-efficient fine-tuning can substantially reduce the computational resources required for model adaptation while retaining comparable classification performance under the experimental conditions of this study. The study also highlights a trade-off between memory efficiency, training time, and predictive performance.

Table of Contents

Table of Contents

1	Introduction	3
2	Background and Related Work	6
3	Dataset and Problem Definition	9
4	Methodology	12
5	Experimental Setup	16
6	Results	19
7	Discussion	24
8	Limitations	27
9	Conclusion	29

1. Introduction

1.1 Context and Motivation

Transformer-based pretrained language models have become an important approach for natural language processing applications. These models are first trained on large text corpora and can subsequently be adapted to specific downstream tasks through fine-tuning. For a classification task such as sentiment analysis, fine-tuning allows a pretrained language representation to be adapted to the characteristics of the target dataset.

The conventional approach is Full Fine-Tuning, in which the parameters of the pretrained model are updated during task-specific training. Although this approach can provide strong predictive performance, it requires optimization of the complete set of trainable model parameters. The resulting memory requirements include not only the model parameters but also gradients and optimizer states associated with those parameters. These requirements become increasingly important when computational resources are limited.

Parameter-efficient fine-tuning (PEFT) provides an alternative approach in which most of the pretrained model remains frozen while only a comparatively small set of additional or modified parameters is optimized. This reduces the number of parameters participating in gradient-based optimization and can consequently reduce the computational resources required for model adaptation.

This study examines two such approaches, Low-Rank Adaptation (LoRA) and Quantized Low-Rank Adaptation (QLoRA), and compares them against conventional Full Fine-Tuning. The comparison is performed using the `distilbert-base-uncased` transformer model on the Stanford IMDB movie-review sentiment classification dataset. The experimental design evaluates both computational efficiency and classification performance.

1.2 Technical Problem

The central technical problem addressed in this case study is the resource cost associated with adapting pretrained transformer models to downstream classification tasks. Updating the complete model can require substantially more trainable parameters and memory than approaches that freeze the pretrained backbone.

The problem can therefore be expressed as a resource–performance trade-off: reducing the number of parameters involved in fine-tuning may lower memory consumption and training time, but may also affect the predictive performance of the resulting model. A meaningful comparison therefore requires the evaluation of both resource-related and task-related metrics.

To investigate this trade-off, the present study compares three fine-tuning strategies under con-

trolled experimental conditions. The measured resource metrics are trainable parameter count, peak GPU memory consumption, and wall-clock training time. Predictive performance is evaluated using held-out test accuracy and F1 score.

1.3 Objectives

The objectives of this case study are:

1. To establish a Full Fine-Tuning baseline using the `distilbert-base-uncased` model for binary sentiment classification.
2. To implement Low-Rank Adaptation (LoRA) as a parameter-efficient alternative to Full Fine-Tuning.
3. To implement Quantized Low-Rank Adaptation (QLoRA) using a 4-bit quantized model representation together with LoRA adapters.
4. To compare Full Fine-Tuning, LoRA, and QLoRA in terms of trainable parameter count, peak GPU memory consumption, and wall-clock training time.
5. To evaluate the classification performance of the three approaches using test accuracy and F1 score.
6. To interpret the resulting trade-offs between computational resource requirements and predictive performance.

1.4 Research Questions

The experimental analysis is guided by the following research questions:

1. **RQ1:** How does parameter-efficient fine-tuning affect the number of trainable parameters compared with Full Fine-Tuning?
2. **RQ2:** How do Full Fine-Tuning, LoRA, and QLoRA compare in terms of peak GPU memory consumption and wall-clock training time?
3. **RQ3:** How much classification performance is retained by LoRA and QLoRA relative to the Full Fine-Tuning baseline?
4. **RQ4:** What trade-offs between computational efficiency and predictive performance are observed among the three approaches?

1.5 Organization of the Report

The remainder of this report is organized as follows. Section II reviews the background and related research on transformer-based fine-tuning and parameter-efficient adaptation. Section III describes the IMDB dataset and defines the classification problem. Section IV presents the methodology used to implement and compare Full Fine-Tuning, LoRA, and QLoRA. Section V describes the experimental configuration and evaluation procedure. Section VI presents the experimental results, while Section VII interprets the observed resource and performance trade-offs. Section VIII discusses the limitations and threats to validity of the study. Finally, Section IX summarizes the findings and concludes the case study.

2. Background and Related Work

2.1 Transformer-Based Transfer Learning

Transformer-based language models use attention mechanisms to construct contextual representations of text. BERT demonstrated that a bidirectional transformer representation pretrained on large-scale textual data could subsequently be adapted to downstream natural language processing tasks [1]. This pretrained-and-adapt paradigm provides a foundation for applying a single language representation to multiple task-specific problems.

In the present study, the pretrained `distilbert-base-uncased` model is used as the base architecture for binary sentiment classification. Rather than training a language model from the beginning, the experiment adapts an existing pretrained representation to the target classification task.

2.2 Full Fine-Tuning

Full Fine-Tuning represents the conventional approach to adapting a pretrained model. During task-specific training, the parameters of the model are updated using the target dataset. This provides the optimization procedure with access to the complete parameter space of the model.

The primary advantage of Full Fine-Tuning is that the entire model can adapt to the characteristics of the downstream task. However, updating the complete parameter set also increases the number of parameters for which gradients and optimizer states must be maintained. Consequently, the computational and memory requirements of the training process can increase with model size.

In this case study, Full Fine-Tuning is therefore used as the baseline against which the two parameter-efficient approaches are evaluated.

2.3 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning (PEFT) methods seek to adapt pretrained models while updating substantially fewer parameters than conventional Full Fine-Tuning. One early approach is the use of task-specific adapter modules, in which most of the original model parameters remain fixed while relatively small additional components are trained [2].

The central motivation behind PEFT is to reduce the computational and storage requirements associated with adapting large pretrained models. Instead of treating every pretrained parameter as trainable, PEFT methods restrict optimization to a smaller set of task-specific parameters or parameter updates.

The present study focuses on two PEFT techniques: Low-Rank Adaptation (LoRA) and Quantized Low-Rank Adaptation (QLoRA).

2.4 Low-Rank Adaptation

Low-Rank Adaptation (LoRA) introduces trainable low-rank matrices into selected layers of a pretrained model while keeping the original model weights frozen [3]. Instead of directly modifying a pretrained weight matrix W_0 , LoRA represents the task-specific update using a low-rank decomposition:

$$W = W_0 + \Delta W \quad (1)$$

where

$$\Delta W = BA. \quad (2)$$

Here, B and A are trainable low-rank matrices, while the original matrix W_0 remains frozen. The rank r of the decomposition is selected to be substantially smaller than the dimensions of the original weight matrix.

This parameterization reduces the number of parameters that must receive gradient updates. In the experimental configuration used in this study, LoRA adapters are applied to the query and value projection layers of the self-attention mechanism with rank $r = 8$.

2.5 Quantized Low-Rank Adaptation

Quantized Low-Rank Adaptation (QLoRA) extends the parameter-efficient approach by combining low-rank adaptation with quantization of the pretrained model weights. QLoRA was introduced as a method for fine-tuning quantized language models while maintaining trainable low-rank adapters [4].

The use of a lower-precision representation reduces the memory required to store the pretrained model parameters. The adapter parameters remain trainable while the quantized base model is kept frozen. In the present experiment, QLoRA uses a 4-bit NormalFloat (NF4) representation together with double quantization and the same rank-8 LoRA configuration used for the standard LoRA experiment.

The resulting approach introduces an additional systems-level trade-off. Quantization can reduce the memory footprint of the base model, but the quantized representation may introduce additional computational overhead during model execution.

2.6 Motivation for the Comparative Study

The preceding approaches address model adaptation using different parameter and resource strategies. Full Fine-Tuning updates the complete model, LoRA restricts updates to low-rank adapter parameters, and QLoRA additionally quantizes the pretrained model representation.

These differences motivate an empirical comparison rather than an evaluation based only on parameter count. A method that uses fewer trainable parameters may reduce memory consumption without necessarily producing the shortest training time, while a small difference in predictive performance may accompany a substantial resource reduction.

Therefore, this case study evaluates the three approaches using both computational and predictive metrics. Trainable parameter count, peak GPU memory, and wall-clock training time are used to characterize resource requirements, while test accuracy and F1 score are used to evaluate downstream classification performance.

3. Dataset and Problem Definition

3.1 Dataset Description

The Stanford Large Movie Review Dataset (IMDB) is used as the benchmark dataset for the sentiment classification task [5]. The dataset consists of movie reviews annotated with binary sentiment labels. Each review is associated with either a positive or a negative sentiment class.

The dataset is suitable for this case study because it provides a text classification problem in which the same pretrained language model can be evaluated under different fine-tuning strategies. The focus of the experiment is therefore not on developing a new classification architecture, but on measuring how different model adaptation strategies affect computational requirements and predictive performance.

3.2 Dataset Source and Availability

The dataset used in the experiment is the Stanford Large Movie Review Dataset. The dataset was accessed programmatically through the dataset loading framework used in the experimental notebook. The original dataset is described by Maas et al. [5].

For the experiments reported in this study, a controlled subset of the available data was used rather than the complete dataset. This reduction was applied to make the experiments feasible within the available computational environment while keeping the comparison between Full Fine-Tuning, LoRA, and QLoRA consistent.

3.3 Dataset Sampling and Splits

The experimental dataset was constructed using a fixed random seed of 42. The training data was reduced to 4,000 samples, while 1,000 additional samples were used for validation. A separate set of 1,000 samples was reserved for final testing.

The resulting dataset organization is shown in Table 1.

Table 1: Dataset split used in the experiments

Split	Samples	Purpose
Training	4,000	Model optimization
Validation	1,000	Evaluation during training
Test	1,000	Final performance evaluation

The training subset contains 2,012 positive and 1,988 negative examples. The fixed random seed ensures that the same experimental subset can be reproduced when the corresponding data preparation procedure is executed again.

3.4 Preprocessing and Tokenization

The text inputs were processed using the tokenizer associated with the `distilbert-base-uncased` pretrained model. Each review was converted into the token representation expected by the transformer model.

A maximum sequence length of 256 tokens was used. Reviews longer than this limit were truncated, while shorter inputs were padded to the required sequence length. The same preprocessing procedure was applied across all three experimental conditions so that differences in the measured results could be attributed to the fine-tuning strategy rather than to different input processing procedures.

3.5 Problem Definition

The task is formulated as a binary text classification problem. Given a movie review represented as a sequence of tokens, the model must predict one of two sentiment classes:

$$y \in \{0, 1\} \quad (3)$$

where the two classes correspond to negative and positive sentiment, respectively.

Let x denote the tokenized representation of a review and let f_θ denote the DistilBERT-based classification model. The objective is to learn model parameters such that

$$f_\theta(x) \rightarrow y \quad (4)$$

where the predicted label corresponds to the sentiment expressed in the input review.

The central experimental problem is therefore to determine how the choice of fine-tuning strategy affects the computational cost and classification performance of the same pretrained model. Full Fine-Tuning, LoRA, and QLoRA are evaluated using the same dataset, preprocessing procedure, and task definition.

3.6 Experimental Objectives

The dataset and task definition establish the basis for a controlled comparison of the three fine-tuning strategies. The experiment therefore considers both resource efficiency and predictive performance.

The primary objectives are:

1. To compare the number of trainable parameters required by Full Fine-Tuning, LoRA,

and QLoRA.

2. To measure the peak GPU memory consumed during training under each fine-tuning strategy.
3. To compare the wall-clock training time of the three approaches.
4. To evaluate classification performance using test accuracy and weighted F1 score.
5. To analyze the trade-off between computational efficiency and predictive performance.

4. Methodology

4.1 Overview of the Experimental Method

The study uses a controlled comparative methodology to evaluate three approaches for adapting the same pretrained transformer model to binary sentiment classification: Full Fine-Tuning, Low-Rank Adaptation (LoRA), and Quantized Low-Rank Adaptation (QLoRA).

The three approaches are evaluated using the same downstream task, dataset splits, preprocessing procedure, number of training epochs, and batch size. The primary methodological difference is the manner in which the pretrained model parameters are represented and updated during training.

Full Fine-Tuning serves as the baseline approach in which the parameters of the pretrained model are updated during task-specific training. LoRA freezes the pretrained parameters and introduces trainable low-rank adapter parameters into selected attention layers. QLoRA follows the same low-rank adaptation strategy while also storing the pretrained model using a 4-bit quantized representation.

The methodology therefore enables the comparison of parameter efficiency, memory consumption, training time, and predictive performance under a common experimental setting.

4.2 Base Model Architecture

The pretrained model used in all experiments is `distilbert-base-uncased`. DistilBERT is a transformer-based language model derived from the BERT architecture and is used here as the common base model for all three fine-tuning strategies.

The selected model contains six transformer layers, twelve attention heads, and a hidden dimension of 768. The base model contains approximately 66.9 million parameters. A task-specific classification head is used to produce the binary sentiment prediction required by the IMDB classification task.

Using the same base architecture for all experiments ensures that the comparison focuses on the fine-tuning strategy rather than on differences between model architectures.

4.3 Full Fine-Tuning

Full Fine-Tuning represents the conventional adaptation strategy used as the baseline in this study. Under this approach, the parameters of the pretrained DistilBERT model and its task-specific components are updated during training.

Let θ denote the complete set of model parameters. The optimization process updates θ using

the task-specific training objective:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) \quad (5)$$

where $\mathcal{L}$ represents the training loss.

Because the complete parameter set is trainable, Full Fine-Tuning requires gradient computation and optimizer-state storage for the model parameters involved in training. This makes it the reference method for evaluating the resource reductions achieved by the parameter-efficient approaches.

In the experimental configuration, Full Fine-Tuning uses the AdamW optimizer with a learning rate of 2×10^{-5} and mixed precision using FP16.

4.4 Low-Rank Adaptation

Low-Rank Adaptation (LoRA) modifies the conventional optimization procedure by freezing the pretrained model weights and introducing trainable low-rank matrices into selected layers [3].

For an original weight matrix W_0 , LoRA represents the updated weight as

$$W = W_0 + \Delta W \quad (6)$$

where the task-specific update is represented by

$$\Delta W = BA. \quad (7)$$

Here, A and B are trainable matrices whose rank is substantially smaller than the dimensions of the original weight matrix. The original matrix W_0 remains frozen during training.

For the experiments in this study, LoRA adapters are applied to the query and value projection modules, identified as `q_lin` and `v_lin`. The adapter configuration uses a rank of

$$r = 8 \quad (8)$$

an LoRA scaling parameter of

$$\alpha = 16, \quad (9)$$

and a dropout probability of 0.10.

Only the LoRA parameters and the task-specific trainable components are optimized, while the pretrained base parameters remain frozen. The LoRA experiment uses AdamW with a learning rate of 5×10^{-4} and FP16 mixed-precision computation.

4.5 Quantized Low-Rank Adaptation

Quantized Low-Rank Adaptation (QLoRA) combines low-rank adaptation with quantization of the pretrained model representation [4]. The objective is to reduce the memory required to store the pretrained parameters while retaining the parameter-efficient optimization procedure of LoRA.

In this study, QLoRA uses a 4-bit NormalFloat (NF4) representation for the pretrained model together with double quantization. The trainable adapter configuration remains consistent with the LoRA experiment:

- LoRA rank: $r = 8$,
- LoRA scaling factor: $\alpha = 16$,
- LoRA dropout: 0.10,
- target modules: `q_lin` and `v_lin`.

The pretrained parameters remain frozen, while the LoRA parameters are updated during training. FP16 is used as the computation precision.

The classification components are kept outside the quantized modules. In the experimental implementation, the `pre_classifier` and `classifier` modules are excluded from quantization using the corresponding module-skip configuration. This ensures that the task-specific classification components remain compatible with the training procedure.

4.6 Optimization Configuration

The optimization settings differ between Full Fine-Tuning and the two parameter-efficient approaches. Full Fine-Tuning uses a lower learning rate because the complete pretrained parameter set is updated, whereas LoRA and QLoRA use a higher learning rate for the smaller trainable parameter set.

The common training configuration consists of three training epochs and a batch size of 16. FP16 computation is used across the three experimental conditions.

The main optimization settings are summarized in Table 2.

Table 2: Fine-tuning configurations used in the experiments

Configuration	Full Fine-Tuning	LoRA	QLoRA
Base model	DistilBERT	DistilBERT	DistilBERT
Trainable base weights	Yes	No	No
LoRA rank	–	8	8
LoRA α	–	16	16
LoRA dropout	–	0.10	0.10
Target modules	–	q_lin, v_lin	q_lin, v_lin
Quantization	None	None	4-bit NF4
Double quantization	No	No	Yes
Optimizer	AdamW	AdamW	AdamW
Learning rate	2×10^{-5}	5×10^{-4}	5×10^{-4}
Epochs	3	3	3
Batch size	16	16	16
Computation	FP16	FP16	FP16

4.7 Evaluation Method

The experimental evaluation considers both computational efficiency and downstream classification performance.

Computational efficiency is assessed using three measurements: trainable parameter count, peak GPU memory consumption, and wall-clock training time. These measurements characterize the amount of model state being optimized, the maximum GPU memory allocated during training, and the elapsed training time, respectively.

Predictive performance is evaluated on the held-out test set using classification accuracy and weighted F1 score. Accuracy measures the proportion of correctly classified reviews, while the weighted F1 score provides a class-weighted measure based on precision and recall.

The combination of resource and predictive metrics allows the three fine-tuning strategies to be compared as complete experimental configurations rather than solely on the number of trainable parameters.

5. Experimental Setup

5.1 Computational Environment

The experiments were conducted using Google Colab Free Tier as the computational environment. The available hardware included an NVIDIA Tesla T4 GPU with approximately 15 GB of VRAM.

The use of a fixed computational environment was intended to keep the comparison between Full Fine-Tuning, LoRA, and QLoRA consistent. All three approaches were executed using the same underlying model, dataset configuration, and general training procedure.

The computational environment is summarized in Table 3.

Table 3: Computational environment used for the experiments

Component	Configuration
Platform	Google Colab Free Tier
GPU	NVIDIA Tesla T4
GPU Memory	Approximately 15 GB VRAM
Base Model	<code>distilbert-base-uncased</code>
Task	Binary sentiment classification
Dataset	Stanford IMDB
Maximum Sequence Length	256 tokens
Random Seed	42

5.2 Dataset Configuration

The same dataset configuration was used for all three fine-tuning experiments. The experimental dataset consisted of 4,000 training samples, 1,000 validation samples, and 1,000 test samples.

The training subset contained 2,012 positive examples and 1,988 negative examples. A fixed random seed of 42 was used during the data preparation procedure to provide a reproducible experimental configuration.

All text inputs were tokenized using the tokenizer associated with `distilbert-base-uncased`. The maximum sequence length was set to 256 tokens, with longer sequences truncated and shorter sequences padded to the required length.

5.3 Training Configuration

Each fine-tuning approach was trained for three epochs using a batch size of 16. The same training duration and batch size were used across the three experimental conditions.

Full Fine-Tuning used AdamW with a learning rate of 2×10^{-5} . LoRA and QLoRA used AdamW with a learning rate of 5×10^{-4} . FP16 computation was used throughout the experiments.

The LoRA configuration used rank $r = 8$, scaling factor $\alpha = 16$, and dropout of 0.10. The adapters were applied to the `q_lin` and `v_lin` modules.

For QLoRA, the pretrained model was represented using 4-bit NF4 quantization with double quantization. The classification-related modules `pre_classifier` and `classifier` were excluded from quantization according to the experimental configuration.

5.4 Experimental Procedure

The experiments followed a common sequence of operations. First, the pretrained DistilBERT model and the corresponding tokenizer were loaded. The dataset was then prepared using the fixed training, validation, and test subsets.

For each fine-tuning strategy, the appropriate model configuration was initialized and training was performed using the specified optimization parameters. The validation subset was used for evaluation during training, while the test subset was reserved for final performance reporting.

The same general procedure was repeated independently for Full Fine-Tuning, LoRA, and QLoRA. This provided a consistent basis for comparing the resource requirements and predictive performance of the three approaches.

5.5 Performance Measurements

The experiment records both computational and predictive measurements.

The number of trainable parameters was recorded to quantify the portion of the model being optimized under each fine-tuning strategy. Peak GPU memory allocation was measured during training using the CUDA memory measurement functionality. Wall-clock training time was recorded to characterize the elapsed time required for each experiment.

After training, the resulting models were evaluated on the held-out test set. Test accuracy and weighted F1 score were recorded as the primary predictive performance metrics.

The resulting measurements are used in Section 6 to compare the three approaches.

5.6 Reproducibility Considerations

A fixed random seed of 42 was used for the dataset preparation procedure. The same model architecture, dataset configuration, sequence length, batch size, number of epochs, and evaluation procedure were maintained across the three experimental conditions.

The experiments were implemented in the project notebook `notebooks/peft_benchmark.ipynb`. The computational environment and experimental configuration described in this section provide the basis for reproducing the reported comparison.

5.7 Code and Repository

The source code, experimental notebook, and supporting files used for this study are available in the project repository:

<https://github.com/DevKumar005/PEFT-Benchmark/tree/main>

The repository contains the implementation of the fine-tuning experiments, data preparation procedures, evaluation code, and supporting analysis artifacts associated with this case study.

6. Results

6.1 Overview of Experimental Results

The experiments compare Full Fine-Tuning, LoRA, and QLoRA using five primary evaluation measures: trainable parameter count, peak GPU memory consumption, wall-clock training time, test accuracy, and weighted F1 score.

The results demonstrate differences in both computational requirements and predictive performance. Full Fine-Tuning provides the baseline for comparison, while LoRA and QLoRA substantially reduce the number of trainable parameters and GPU memory required during training.

6.2 Parameter Efficiency

The number of trainable parameters provides a direct measure of how much of the model is being optimized during task-specific training.

Full Fine-Tuning uses 66,955,010 trainable parameters. In contrast, both LoRA and QLoRA use 739,586 trainable parameters. Thus, the two parameter-efficient approaches optimize only a small fraction of the parameters involved in the Full Fine-Tuning configuration.

The comparison is illustrated in Figure 1.

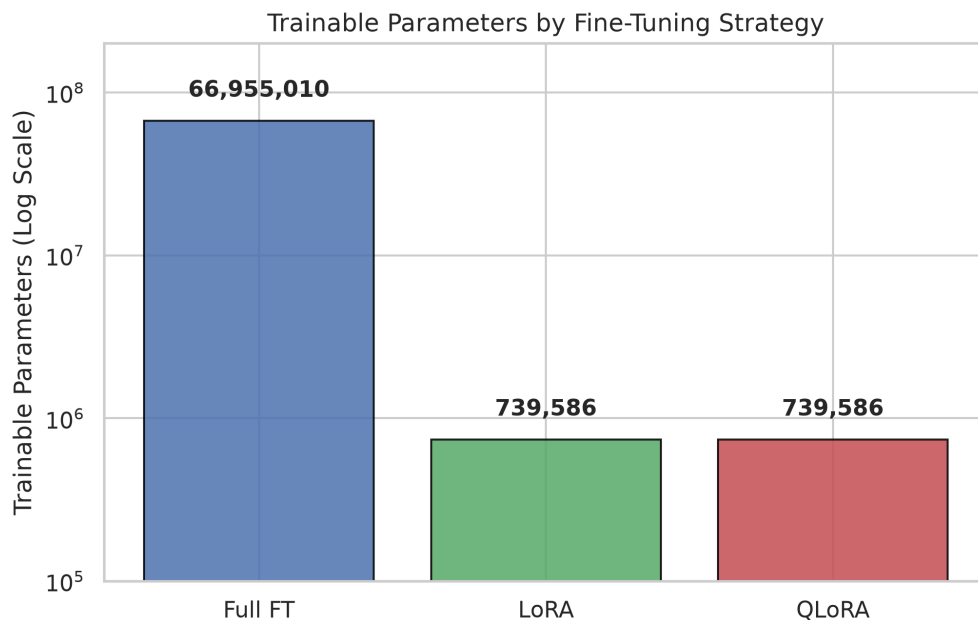


Figure 1: Trainable parameter count for Full Fine-Tuning, LoRA, and QLoRA. The parameter count is shown on a logarithmic scale.

The LoRA configuration reduces the number of trainable parameters by approximately 98.90%

relative to Full Fine-Tuning. QLoRA uses the same number of trainable parameters as LoRA in this experiment, because both approaches use the same low-rank adapter configuration. The difference between the two methods therefore arises primarily from the quantized representation of the pretrained model rather than from the number of trainable adapter parameters.

6.3 Training Loss

Training loss was recorded throughout the training process to observe the optimization behavior of the three approaches.

Figure 2 presents the training loss across the recorded training steps.

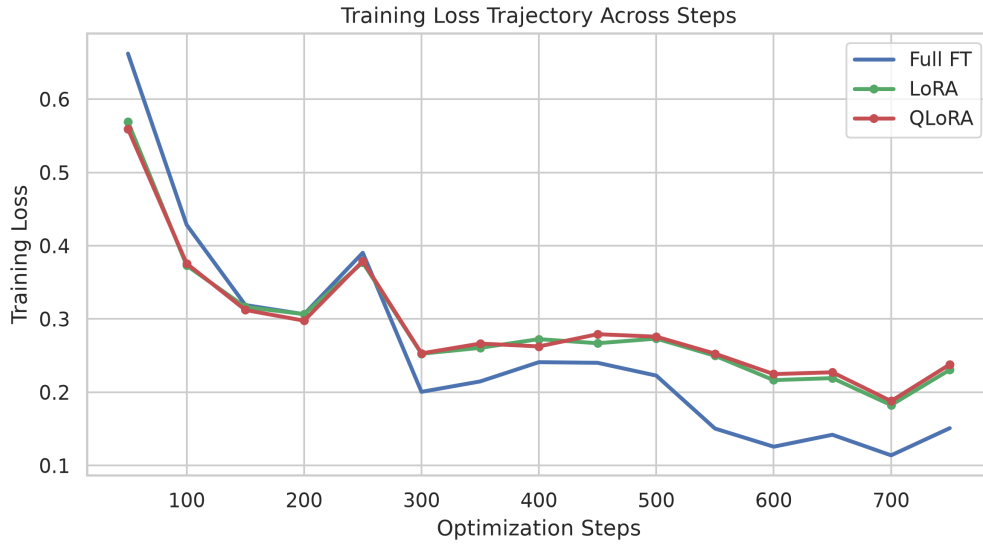


Figure 2: Training loss across the recorded training steps for Full Fine-Tuning, LoRA, and QLoRA.

At the end of training, the reported terminal training losses were approximately 0.151 for Full Fine-Tuning, 0.231 for LoRA, and 0.238 for QLoRA. The lower terminal loss observed for Full Fine-Tuning indicates that, under the experimental configuration, the fully trainable model achieved a lower training loss on the training data.

The loss curves also provide a view of the optimization process rather than only the final classification metrics. Differences in the curves therefore help characterize how the three approaches adapted to the training data.

6.4 GPU Memory Consumption

Peak GPU memory consumption was measured during training to evaluate the memory requirements of each fine-tuning strategy.

Full Fine-Tuning reached a peak memory allocation of 1,720.34 MB. LoRA reduced this value to 995.70 MB, while QLoRA recorded the lowest peak allocation at 966.32 MB.

The memory and training-time comparison is shown in Figure 3.

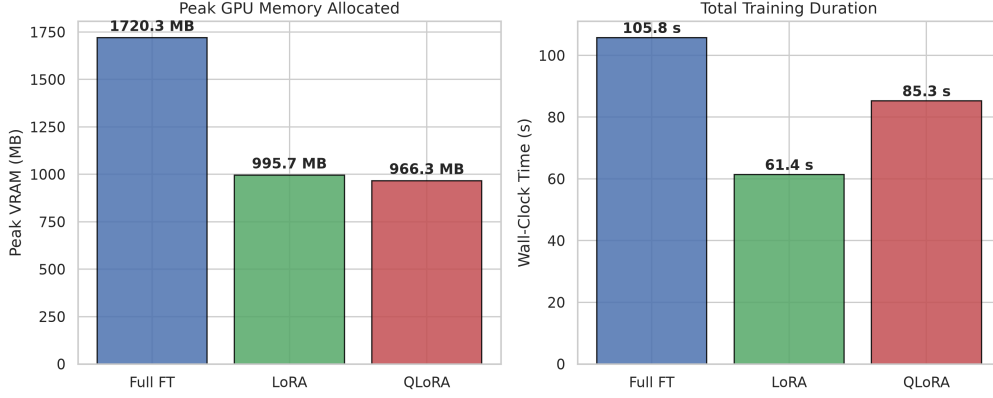


Figure 3: Comparison of peak GPU memory consumption and wall-clock training time across the three fine-tuning strategies.

Relative to Full Fine-Tuning, the measured peak GPU memory consumption was reduced by approximately 42.12% with LoRA. QLoRA produced a further reduction in peak memory under the experimental configuration.

These results show the memory-saving effect of freezing the pretrained parameters and, in the case of QLoRA, representing the pretrained model using a lower-precision quantized format.

6.5 Training Time

The measured wall-clock training times were 105.76 seconds for Full Fine-Tuning, 61.40 seconds for LoRA, and 85.26 seconds for QLoRA.

LoRA therefore required less measured training time than Full Fine-Tuning in this experiment. Its training time was reduced by approximately 41.94% relative to the Full Fine-Tuning baseline.

QLoRA, despite achieving the lowest measured peak GPU memory consumption, required more training time than LoRA. This indicates that lower memory consumption did not directly translate into the shortest execution time. The additional computational operations associated with the quantized representation provide a possible systems-level explanation for this difference within the experimental configuration.

6.6 Classification Performance

The final predictive performance was evaluated using test accuracy and weighted F1 score. The results are summarized in Table 4.

Table 4: Summary of experimental results

Method	Trainable Params.	Peak VRAM (MB)	Time (s)	Accuracy	F1
Full Fine-Tuning	66,955,010	1,720.34	105.76	88.60%	0.8860
LoRA	739,586	995.70	61.40	87.20%	0.8720
QLoRA	739,586	966.32	85.26	86.70%	0.8670

Full Fine-Tuning achieved a test accuracy of 88.60% and a weighted F1 score of 0.8860. LoRA achieved 87.20% accuracy and a weighted F1 score of 0.8720, while QLoRA achieved 86.70% accuracy and a weighted F1 score of 0.8670.

The predictive performance differences are relatively small compared with the differences in trainable parameter count and memory consumption. This indicates that the parameter-efficient approaches were able to retain much of the classification performance of the baseline while substantially reducing the resources required for adaptation.

6.7 Accuracy–Memory Trade-off

The relationship between predictive performance and memory consumption is illustrated in Figure 4.

Full Fine-Tuning provides the highest measured test accuracy in the experiment but also has the highest peak GPU memory consumption. LoRA occupies an intermediate position in terms of memory consumption while maintaining a test accuracy relatively close to the Full Fine-Tuning result.

QLoRA achieves the lowest measured memory consumption but also records the lowest test accuracy among the three configurations. The figure therefore provides a direct visualization of the trade-off between predictive performance and computational resource consumption observed in the experiment.

6.8 Overall Results

Table 4 demonstrates that the three fine-tuning strategies produce different resource-performance profiles.

Full Fine-Tuning updates the largest number of parameters and records the highest peak GPU memory consumption and longest measured training time. LoRA substantially reduces the

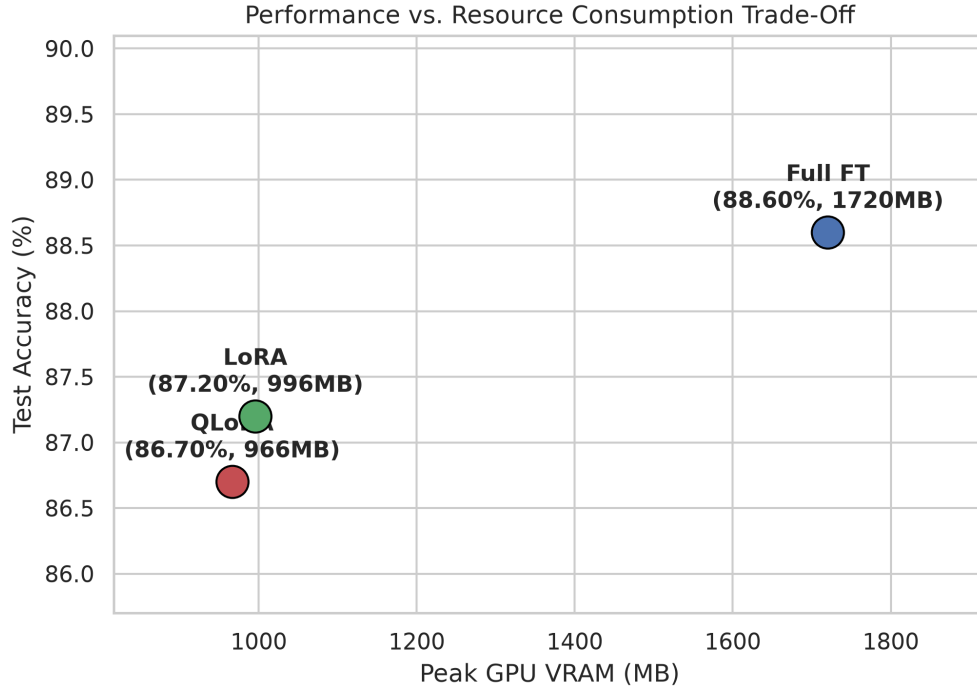


Figure 4: Test accuracy versus peak GPU memory consumption for the three fine-tuning strategies.

trainable parameter count, memory consumption, and training time while maintaining a test accuracy of 87.20%.

QLoRA achieves the lowest measured peak GPU memory consumption, but its measured training time is higher than LoRA and its test accuracy is slightly lower under the experimental conditions.

These results provide the empirical basis for the discussion of resource efficiency and predictive performance presented in the next section.

7. Discussion

7.1 Interpretation of Parameter Efficiency

The experimental results demonstrate a substantial difference in the number of parameters that must be updated during task-specific training. Full Fine-Tuning updates the complete trainable parameter set, whereas LoRA and QLoRA restrict optimization to a much smaller set of adapter parameters.

In the present experiment, LoRA and QLoRA each use 739,586 trainable parameters compared with 66,955,010 for Full Fine-Tuning. This represents a reduction of approximately 98.90% in the number of trainable parameters.

This reduction is significant from a parameter-efficiency perspective because the parameter count directly reflects the portion of the model that participates in gradient-based optimization. The results therefore demonstrate the central principle of parameter-efficient fine-tuning: a downstream task can be learned without updating the complete pretrained parameter set.

7.2 Memory Efficiency

The memory measurements provide a second perspective on the resource requirements of the three approaches.

Full Fine-Tuning recorded a peak GPU memory consumption of 1,720.34 MB, compared with 995.70 MB for LoRA and 966.32 MB for QLoRA. LoRA therefore reduced the measured peak memory consumption by approximately 42.12% relative to Full Fine-Tuning.

The lower memory requirement observed for LoRA is consistent with its frozen-base-model configuration. Since the pretrained parameters are not updated, the optimization process does not require the same training-state representation for the complete model.

QLoRA produced the lowest measured peak memory consumption in the experiment. In addition to using LoRA adapters, QLoRA stores the pretrained model using a 4-bit quantized representation. The combination of parameter-efficient optimization and quantized storage therefore resulted in the smallest measured memory footprint among the three configurations.

7.3 Training-Time Trade-off

The measured training times reveal that memory efficiency and execution time are not equivalent objectives.

LoRA required 61.40 seconds, compared with 105.76 seconds for Full Fine-Tuning. This corresponds to a measured reduction of approximately 41.94% in wall-clock training time relative

to the baseline.

QLoRA required 85.26 seconds, which was longer than LoRA despite using less peak GPU memory. This result demonstrates that reducing the memory representation of the pretrained model does not necessarily produce the shortest execution time.

Within the experimental implementation, the quantized representation introduces additional operations associated with handling the quantized model during computation. Consequently, QLoRA achieved lower measured memory consumption than LoRA while requiring more wall-clock time.

This distinction is important when evaluating parameter-efficient methods because computational efficiency has multiple dimensions. Memory footprint and execution time should therefore be considered separately rather than treated as a single measure of efficiency.

7.4 Predictive Performance

The classification results show that all three approaches achieved substantial predictive performance on the held-out IMDB test set.

Full Fine-Tuning achieved 88.60% test accuracy and a weighted F1 score of 0.8860. LoRA achieved 87.20% accuracy and a weighted F1 score of 0.8720, while QLoRA achieved 86.70% accuracy and a weighted F1 score of 0.8670.

The results indicate that the parameter-efficient approaches did not cause a large reduction in classification performance under the conditions of this experiment. LoRA remained relatively close to the Full Fine-Tuning result while using substantially fewer trainable parameters and less GPU memory.

QLoRA produced a slightly lower test accuracy and F1 score than LoRA. Therefore, the additional memory reduction obtained through quantization was accompanied by a small difference in predictive performance in this particular experimental configuration.

These observations should be interpreted as empirical findings for the selected model, dataset, and training configuration rather than as universal properties of the three fine-tuning strategies.

7.5 Relationship Between Training Loss and Test Performance

The training-loss results provide additional context for the classification results.

Full Fine-Tuning reached a lower terminal training loss than both LoRA and QLoRA. The reported terminal losses were approximately 0.151 for Full Fine-Tuning, 0.231 for LoRA, and 0.238 for QLoRA.

The lower training loss indicates that Full Fine-Tuning was able to fit the training data more

closely under the experimental configuration. However, training loss alone does not determine generalization performance. The held-out test metrics provide the more direct measure of how the resulting models performed on unseen examples.

The relatively small differences in test accuracy and weighted F1 between the approaches therefore show why both optimization behavior and held-out evaluation are useful when analyzing fine-tuning strategies.

7.6 Overall Resource–Performance Trade-off

The experimental results demonstrate that the three methods occupy different points in the resource–performance space.

Full Fine-Tuning provides the reference configuration with the largest trainable parameter set and the highest measured test accuracy. LoRA substantially reduces the number of trainable parameters, GPU memory consumption, and measured training time, while maintaining a test accuracy relatively close to the baseline.

QLoRA achieves the lowest measured peak GPU memory consumption in the experiment. However, its measured training time is higher than LoRA and its test performance is slightly lower.

The results therefore indicate that the choice of fine-tuning strategy involves multiple competing considerations. A comparison based solely on predictive accuracy would not capture the substantial differences in computational requirements, while a comparison based solely on memory consumption would not capture the differences in execution time and predictive performance.

7.7 Implications for Parameter-Efficient Fine-Tuning

The findings support the practical motivation for evaluating parameter-efficient fine-tuning methods under resource constraints. LoRA provides a substantial reduction in the number of trainable parameters and measured GPU memory while maintaining similar classification performance to Full Fine-Tuning in this experiment. QLoRA extends this approach by further reducing the measured memory requirement through quantization. However, the results also demonstrate that quantization introduces a computational trade-off, as QLoRA required more training time than LoRA under the tested configuration.

Consequently, parameter efficiency should be evaluated using a combination of model parameters, memory consumption, execution time, and predictive performance. The results of this study provide an empirical comparison of these dimensions for the selected DistilBERT and IMDB configuration.

8. Limitations

8.1 Model Scope

The experiments in this study are conducted using `distilbert-base-uncased` as the underlying pretrained model. Although this provides a consistent basis for comparing Full Fine-Tuning, LoRA, and QLoRA, the results are limited to the selected model architecture.

Different transformer architectures and model sizes may exhibit different relationships between parameter efficiency, memory consumption, training time, and predictive performance. Therefore, the findings of this study should not be interpreted as directly representative of all pre-trained language models.

8.2 Random Seed and Experimental Variability

The experimental data preparation uses a single random seed, specifically seed 42. Using one seed provides a reproducible experimental configuration, but it does not measure the variation that may occur across different random initializations or data subsets.

Repeated experiments using multiple random seeds could provide a more robust estimate of the variability of the measured performance. Such repeated trials were outside the scope of the present experiment.

8.3 LoRA Adapter Placement

The LoRA and QLoRA configurations apply adapters only to the `q_lin` and `v_lin` projection modules. Other possible adapter placements and configurations were not evaluated.

Consequently, the reported LoRA and QLoRA results describe the specific adapter configuration used in this study rather than the full range of possible LoRA configurations.

8.4 Dataset Subsampling

The experiments use controlled subsets of the IMDB dataset rather than the complete available dataset. The training, validation, and test sets contain 4,000, 1,000, and 1,000 samples, respectively.

This reduced experimental scale makes the comparison feasible within the available computational environment, but it also limits the extent to which the results can represent performance on the complete dataset.

8.5 Computational Environment

The experiments were conducted on an NVIDIA Tesla T4 GPU through Google Colab Free Tier. Consequently, the measured wall-clock training times and GPU memory values are specific to the hardware, software environment, and execution conditions used during the experiments.

Changes in GPU architecture, software versions, library implementations, system load, or other execution conditions may produce different absolute measurements. The reported resource measurements should therefore be interpreted as measurements of this experimental environment rather than universal hardware benchmarks.

8.6 Limited Evaluation Metrics

The study evaluates the models using trainable parameter count, peak GPU memory, wall-clock training time, test accuracy, and weighted F1 score.

These metrics provide a useful basis for comparing computational efficiency and classification performance, but they do not capture every aspect of model adaptation. For example, the experiment does not evaluate model storage requirements for multiple downstream tasks, inference latency, energy consumption, or performance across additional datasets.

8.7 Implications of the Limitations

The limitations described above define the scope within which the experimental findings should be interpreted. The study provides a controlled comparison of three fine-tuning strategies under a specific DistilBERT, IMDB, and hardware configuration.

The results establish empirical differences within this configuration, but broader conclusions would require experiments across additional model architectures, datasets, random seeds, adapter placements, and computational environments.

9. Conclusion

This case study presented an empirical comparison of three approaches for adapting a pre-trained transformer model to a binary sentiment classification task: Full Fine-Tuning, Low-Rank Adaptation (LoRA), and Quantized Low-Rank Adaptation (QLoRA). The experiments used `distilbert-base-uncased` on a controlled subset of the Stanford IMDB dataset and evaluated both computational requirements and predictive performance.

The experimental results demonstrated substantial differences in parameter efficiency and resource consumption. Full Fine-Tuning updated 66,955,010 parameters, whereas LoRA and QLoRA updated only 739,586 parameters. This corresponds to an approximately 98.90% reduction in trainable parameters for the parameter-efficient approaches.

The resource measurements also showed substantial differences between the three configurations. Full Fine-Tuning recorded a peak GPU memory consumption of 1,720.34 MB and a wall-clock training time of 105.76 seconds. LoRA reduced these measurements to 995.70 MB and 61.40 seconds, respectively. QLoRA achieved the lowest measured peak GPU memory consumption at 966.32 MB, while requiring 85.26 seconds of training time.

In terms of predictive performance, Full Fine-Tuning achieved a test accuracy of 88.60% and a weighted F1 score of 0.8860. LoRA achieved 87.20% accuracy and a weighted F1 score of 0.8720, while QLoRA achieved 86.70% accuracy and a weighted F1 score of 0.8670.

Taken together, the results show that parameter-efficient fine-tuning substantially reduced the computational resources required in the experimental configuration while maintaining classification performance relatively close to the Full Fine-Tuning baseline. The comparison also demonstrated that memory reduction and training-time reduction are distinct considerations: QLoRA achieved the lowest measured memory consumption but required more training time than LoRA.

The findings are limited to the selected DistilBERT model, IMDB dataset subset, single random seed, adapter placement, and computational environment described in this study. Broader evaluation across different models, datasets, random seeds, and adapter configurations would be required to determine whether the observed relationships generalize beyond the present experimental setup.

Overall, the study demonstrates the value of evaluating fine-tuning strategies using multiple computational and predictive metrics rather than relying on a single measure. Parameter count, GPU memory, training time, and classification performance together provide a more complete characterization of the trade-offs involved in adapting pretrained transformer models.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.
- [2] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for nlp,” in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 2790–2799.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” *arXiv preprint arXiv:2305.14314*, 2023.
- [5] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 142–150.